

System Requirements Specification (SRS)

Version 0.1

Revision History

Date	Version	Description	Author
2025 - 06 - 27	0.1	Initial template creation and first content draft what's highlighted are things I'm not sure about	Skander Hellal

Table of Content

1. Introduction	5
1.1. Objective	5
1.2 Definitions, Acronyms, and Abbreviations	5
1.3. Overview of the document	6
2. Global Description	6
2.1 User Characteristics	6
2.2. Interfaces	7
2.2.1. User Interfaces	7
2.2.2. Physical Interfaces	7
2.2.3. Software Interfaces	7
2.2.4. Communication Interfaces	8
2.3. General Constraints	8
3. Functional Requirements	9
3.1 Data Acquisition – Live and Offline	9
3.1.1 Live Data Acquisition	9
3.1.2 Offline Data Mode	9
3.2 Channel Selection	9
3.3 EMG Signal Visualization	10
3.4 Feature Extraction	10
3.5 Chat Logging (Event Log)	10
3.5.1 Event Logging	10
3.5.2 Interaction with Embedded AI Models	10
3.7 Touchscreen Interaction and Accessibility	11
3.8 Data Export and Session Replay	11
3.9 Error Handling and Alerts	11
4. Non functional Requirements	12
4.1 Usability	12
4.1.1 Interaction with the Chat and AI Module	12
4.1.2 Channel Selection and Visualization Setup	12
4.1.3 File Loading and Setup Time	12
4.2 Reliability	12
4.2.1 Fault Handling	12
4.3 Performance	12
4.3.1 Data Refresh and Plotting	12
4.3.2 Thread Scheduling	13
4.4 Maintainability	13
4.4.1 Code Standards and Structure	13
4.4.2 Modularity	13

4.4.3 Logging and Debugging	13
4.5 Design Constraints	13
4.5.1 Programming Language and Platform	13
4.5.2 Libraries and Frameworks	13
4.6 Security	13
4.6.1 Local Data Protection	13
4.6.2 AI Model Isolation	14
4.7 User Documentation and Help System Requirements	14
4.7.1 Offline Documentation	14
4.7.2 Touch Screen Usability Hints	14
4.7.3 Error Messages	14
4.8 Applicable Standards	14
4.8.1 Coding Standards	14
4.8.2 UI Accessibility Guidelines	14
4.8.3 Biomedical Signal Processing Conventions	14
4.8.4 Data Format Standards	14

System Requirements Specification (SRS)

1. Introduction

1.1. Objective

The Software Requirements Specification (SRS) defines the intended functionality and external behavior of the application. It outlines both functional and non-functional requirements, design constraints, and other essential criteria that provide a comprehensive and unambiguous description of the software system to be developed.

1.2 Definitions, Acronyms, and Abbreviations

EMG (Electromyography): A method of recording the electrical activity of muscles using surface or intramuscular electrodes.

GUI (Graphical User Interface): The visual interface that users interact with, composed of buttons, dialogs, graphs, and other visual elements.

AI (Artificial Intelligence): Refers to an embedded or local model capable of assisting the user in interpreting EMG data or responding to system queries.

Python: A high-level, interpreted programming language used to develop the entire software system. Known for its readability and extensive library support, it enables rapid development of graphical interfaces, data processing pipelines, and real-time signal analysis.

RMS (Root Mean Square): A measure of the magnitude of the EMG signal, often used in feature extraction.

ZC (Zero Crossing): A time-domain EMG feature that counts the number of times the signal crosses the zero axis.

MNF (Mean Frequency): A frequency-domain EMG metric indicating the average frequency in the power spectrum.

MDF (Median Frequency): The frequency that divides the EMG power spectrum into two regions of equal power.

MAV (Mean Absolute Value): The average of the absolute values of the EMG signal; used as a simple amplitude feature.

WL (Waveform Length): A time-domain feature representing the cumulative length of the waveform over time.

WAMP (Willison Amplitude): A time-domain feature representing the number of times the absolute difference between consecutive signal values exceeds a threshold.

IEMG (Integrated EMG): The cumulative area under the EMG waveform over a defined window.

VAR (Variance): A measure of signal variability over time.

LOG: Logarithmic transformation of EMG amplitude data, used for normalization or scaling.

PSD (Power Spectral Density): A representation of signal power as a function of frequency, used in EMG frequency analysis.

PyQt5: A Python library for building graphical user interfaces using the Qt framework.

PyQtGraph: A fast plotting and data visualization library compatible with PyQt5.

NumPy: A Python library for numerical computing, especially for arrays and matrices.

SciPy: A scientific computing library in Python, often used for signal processing.

neurokit2: A Python toolbox for neurophysiological signal processing, including EMG.

pysiology: A Python library that includes EMG-specific feature extraction functions.

.npz: A compressed file format from NumPy used to store one or more arrays on disk.

.txt: A plain text file format used for storing logs and exported messages.

TTL (Transistor-Transistor Logic): A digital signal used for timing and synchronization between devices.

QThread: A threading class provided by PyQt5 to run background tasks without blocking the user interface.

Sample Rate: The number of data samples recorded per second; fixed at 220 Hz in this system.

Buffer: A temporary data structure that stores a sliding window of real-time samples for visualization or processing.

1.3. Overview of the document

This document describes the functional and non-functional requirements of the EMG signal analysis and visualization system to be deployed on a Raspberry Pi. It is intended to serve as a reference for developers and future maintainers.

The structure of the document is as follows:

- **Section 1** presents the purpose, scope, terminology, and structure of the document.
- **Section 2** outlines the general system description, including user characteristics, hardware/software interfaces, constraints, assumptions, and dependencies.
- **Section 3** details all functional requirements, organized by feature group and labeled as necessary or optional where applicable.
- **Section 4** includes non-functional requirements such as performance, usability, maintainability, design constraints, security, and applicable standards.

2. Global Description

The software is designed to acquire, process, and visualize electromyographic (EMG) signals in real time using embedded systems such as Arduino and Raspberry Pi. It extracts physiological features from the raw data, detects significant signal variations, and dynamically adjusts stimulation parameters for adaptive neuromuscular rehabilitation. The system includes a user-friendly graphical interface and supports integration with advanced machine learning models for intelligent signal interpretation.

2.1 User Characteristics

The system is designed for two main user groups with different technical backgrounds and usage goals:

2.2. Interfaces

2.2.1. User Interfaces

The primary users of the software are biomedical engineers, rehabilitation researchers, and healthcare professionals working with neuromuscular disorders. These users typically have a basic to advanced understanding of biosignal acquisition and analysis. Secondary users may include developers involved in assistive technology, as well as patients or caregivers interacting with simplified interfaces for home-based rehabilitation. Depending on the interface mode used, technical expertise may range.

2.2.2. Physical Interfaces

Arduino-compatible microcontrollers: Acquire electromyographic (EMG) signals in real time via analog input pins.

Raspberry Pi: Serves as the central processing and control unit. It handles data acquisition, processing, real-time visualization, and local storage.

Touchscreen display: Connected to the Raspberry Pi to provide an interactive graphical interface for real-time monitoring, channel selection, zooming, and other user controls.

EMG sensors: Capture analog muscle activity signals for processing and visualization.

TTL pulse generators: Insert precise temporal markers for multi-sensor synchronization.

2.2.3. Software Interfaces

The software system integrates and depends on the following major components and libraries:

- **PyQt5:** Used for the graphical user interface (GUI). It provides dialog windows, toolbars, buttons, checkboxes, and layout management.
- **PyQtGraph:** Handles high-speed real-time data plotting for EMG signal visualization.
- **NeuroKit2** and **physiology:** Libraries for advanced EMG feature extraction (MAV, RMS, WL, ZC, etc.).
- **NumPy / SciPy:** Used for numerical computations and signal processing (including PSD, MNF, MDF calculations). Welch's method has been patched to correct window naming inconsistencies.
- **Custom Modules:**
 - **GraphWidget:** Visualizes live or pre-recorded EMG signals.
 - **ChatWidget:** Provides a logging interface to track system events.
 - **StartupDialog:** User input window to choose App mode.
 - **LiveMode:** Handles real-time data streaming from Arduino devices.
 - **DataThread:** Handles real-time data buffering and emission.
- **File I/O:**
 - Supports loading .npz files containing EMG data.
 - Saves logs in .txt format for user review.

2.2.4. Communication Interfaces

The system communicates with external hardware through the following interfaces:

- **USB Serial Communication**

The primary method for real-time EMG data transfer is a USB connection between the Arduino and the Raspberry Pi. The Arduino streams sensor data via a serial protocol, which is received and processed on the Raspberry Pi. The communication is managed using the PyFirmata protocol.

- **Local File System (Disk I/O)**

The application reads pre-recorded EMG data from .npz files and can export log data as .txt files. This interaction occurs through standard file dialog operations in the GUI.

- **Touchscreen Input (NOT SURE HOW SCREEN IS CONNECTED)**

2.3. General Constraints

The software is designed to run on resource-limited hardware, specifically a Raspberry Pi with constrained memory and processing power. The following general constraints must be considered:

- **Memory Constraints**

- The system must operate within 1–2 GB of RAM, typical for Raspberry Pi models used in embedded deployments.
- Real-time EMG data buffering and plotting are limited to short time windows (last 2–5 seconds) to avoid memory overflow.
- Feature extraction modules must process data in fixed-length segments and avoid storing large arrays in memory.

- **CPU Usage**

- All real-time threads (acquisition, plotting, computation) must be lightweight to prevent GUI freezing or frame drops.
- Heavy processing (full-feature extraction, spectral analysis...) is done periodically rather than continuously.

- **Storage**

- Local data logging is limited to small .txt or .npz files. Temporary files must be managed to avoid local storage saturation.
- External storage (USB drive or cloud sync) may be considered for long-term session logging.

- **Display Resolution and UI Layout**

- The graphical interface must remain responsive and readable on a 7 inch screen
- UI elements are designed using scalable layouts to accommodate variable screen sizes

without compromising usability.

- Power Supply
 - The system must remain functional on a standard Raspberry Pi power source. Components and processes should not introduce excessive current draw.
- No Dependency on High-End GPUs
 - All signal processing and ML models used must be compatible with CPU-only environments, as Raspberry Pi hardware lacks discrete GPUs.

3. Functional Requirements

All functional requirements listed in this section are necessary for the proper functioning of the system and must be fully implemented in the scope of this project.

3.1 Data Acquisition – Live and Offline

Scope: Raspberry Pi with touchscreen interface

3.1.1 Live Data Acquisition

- **3.1.1.1** The system must establish a serial connection with the Arduino via USB.
- **3.1.1.2** The system must read EMG signals in real-time at a fixed sampling rate(220 Hz).
- **3.1.1.3** The system must allow live data to be visualized in real time on the graphical interface.
- **3.1.1.4** The system must allow the data stream to be paused and resumed by the user.
- **3.1.1.5** The system must log events such as start/stop or connection errors in the chat window.

3.1.2 Offline Data Mode

- **3.1.2.1** The system must allow users to load .npz files containing pre-recorded EMG data.
- **3.1.2.2** The system must extract and display available channels from the file.
- **3.1.2.3** The system must continuously loop through the data to simulate real-time playback.
- **3.1.2.4** The system must display a message in the chat when a file is successfully loaded.

3.2 Channel Selection

Scope: All data modes

- **3.2.1.1** The system must provide a dialog for selecting which EMG channels to visualize.
- **3.2.1.2** The system must support selecting or deselecting all channels via a "Select All" checkbox.

- **3.2.1.3** The system must update the graph and log the selected channels in the chat upon confirmation.

3.3 EMG Signal Visualization

Scope: Live and offline data modes

- **3.3.1.1** The system must display real-time or simulated EMG signals on a scrolling plot.
- **3.3.1.2** The system must label each signal with the corresponding channel number.
- **3.3.1.3** The system must allow zooming and panning on the visualized signals.
- **3.3.1.4** The system must update plots at a regular refresh rate (~20 Hz) without freezing.

3.4 Feature Extraction

Scope: Offline mode

- **3.4.1.1** The system must allow the user to trigger feature extraction manually using a button.
- **3.4.1.2** The system must divide the EMG signal into fixed-length segments (10 seconds).
- **3.4.1.3** The system must compute the following features per segment: MAV, RMS, WL, ZC, IEMG, WAMP, VAR, LOG, MNF, MDF.
- **3.4.1.4** The system must display the average of each feature in the chat window.

3.5 Chat Logging (Event Log)

Scope: Entire application

3.5.1 Event Logging

- **3.5.1.1** The system must include a chat window to display logs of system actions, such as file openings, feature extractions, and device connections.
- **3.5.1.2** Each log entry must be time stamped and displayed in chronological order.
- **3.5.1.3** The system must allow users to export the full chat log as a .txt file.

3.5.2 Interaction with Embedded AI Models

- **3.5.2.1** The chat window must allow the user to enter text prompts for querying or interpreting EMG-related data using an embedded AI model.
- **3.5.2.2** The AI model must respond contextually within the same chat interface, maintaining conversation history.

- **3.5.2.3** The system must clearly distinguish between system-generated logs and AI-generated responses.
- **3.5.2.4** If no model is available or if an error occurs, the system must notify the user via the chat interface and log the event.

3.6 Machine Learning Analysis

3.7 Touchscreen Interaction and Accessibility

Scope: Raspberry Pi with touchscreen

- **3.7.1.1** The system must allow all GUI elements (buttons, dialogs, file pickers) to be accessible via touchscreen input.
- **3.7.1.2** The interface must support a minimum screen resolution of 1280x 720 pixels without clipping.
- **3.7.1.3** Button sizes and font styles must remain readable and usable on small screens.
- **3.7.1.4** The system must respond to single-tap input for all critical functions (open file, start stream, extract features...).

3.8 Data Export and Session Replay

Scope: All modes

- **3.8.1.1** The system must allow users to export chat logs as .txt files via a save dialog.
- **3.8.1.2** The system must support the loading of pre-recorded EMG sessions in .npz format.
- **3.8.1.3** When a file is loaded, the system must allow for continuous looping playback to simulate a live session.
- **3.8.1.4** The system must preserve data format integrity when saving logs or processing outputs.

3.9 Error Handling and Alerts

Scope: Entire application

- **3.9.1.1** The system must detect and alert the user if no Arduino is detected on the expected port.
- **3.9.1.2** The system must show a critical error message if a .npz file fails to load or is corrupted.
- **3.9.1.3** If the user attempts to extract features with no channels selected, the system must display a warning.
- **3.9.1.4** Any system message must be logged to the chat interface with a timestamp, including connection status, mode transitions, and user actions.
- **3.9.1.5** The system must ensure the graceful termination of all background threads upon closing the application.

4. Non functional Requirements

4.1 Usability

4.1.1 Interaction with the Chat and AI Module

- Users should be able to ask a question and receive a response from the AI within 5 to 20 seconds under normal processing load.
- Interactions should feel conversational, maintaining continuity over multiple exchanges.
- The interface must clearly differentiate between system logs and AI-generated messages.

4.1.2 Channel Selection and Visualization Setup

- Selecting and confirming a set of EMG channels should take no more than 30 seconds, including loading the updated plots.
- All interaction should be possible via touchscreen taps, without the need for a physical keyboard or mouse.

4.1.3 File Loading and Setup Time

- Opening a .npz file and initializing data visualization should take under 10 seconds for files up to 1 minute in length.
- The system must alert the user within 2 seconds if the file is invalid or missing expected EMG data.

4.2 Reliability

4.2.1 Fault Handling

- If the Arduino or EMG input is disconnected, the system must display an error message and stop data acquisition.
- The system must ensure the chat log and GUI remain responsive during errors or interruptions.

4.3 Performance

4.3.1 Data Refresh and Plotting

- The visualization should refresh at a minimum of 20 frames per second to appear smooth to the user.
- Feature extraction from 10 segments of EMG data must complete in under 2 seconds per channel on a Raspberry Pi.

4.3.2 Thread Scheduling

- Data acquisition threads must maintain a sampling rate of 220 Hz, compensating for processing

delays using buffering.

4.4 Maintainability

4.4.1 Code Standards and Structure

- The codebase must follow standard Python coding conventions for readability and maintainability.
- All modules and classes must include inline comments where applicable.
- Function names, variables, and file structures must use consistent and descriptive naming conventions.

4.4.2 Modularity

- The system must separate core functions into independent modules (e.g., data acquisition, visualization, AI communication) to allow easy testing and updates.

4.4.3 Logging and Debugging

- The system must include clear error messages and logging to assist in debugging and issue resolution.

4.5 Design Constraints

4.5.1 Programming Language and Platform

- The application must be developed entirely in Python (v3.12 or above).
- It must run on a Raspberry Pi without requiring internet access during use.

4.5.2 Libraries and Frameworks

- The system must rely on PyQt5 for GUI design and PyQtGraph for signal visualization.
- Core functionality depends on libraries such as NumPy, SciPy, NeuroKit2, and pypots for EMG signal processing.
- External machine learning models must be integrated.

4.6 Security

4.6.1 Local Data Protection

- All data must remain local to the device and not be transmitted over a network.
- The system must not store any sensitive user data or identifiers in saved files.

4.6.2 AI Model Isolation

- Machine learning models must run locally and not make external API calls during operation.
- The AI interface must not allow execution of arbitrary commands or scripts.

4.7 User Documentation and Help System Requirements

4.7.1 Offline Documentation

- A brief offline user guide must be made available with instructions for setup, file formats, and basic troubleshooting.

4.7.2 Touch Screen Usability Hints

- Touchscreen-specific instructions must be documented in the user guide

4.7.3 Error Messages

- All error messages shown to the user must be self-explanatory and provide guidance on how to resolve the issue.

4.8 Applicable Standards

4.8.1 Coding Standards

- The code must follow the PEP 8 Python style guide for readability and consistency.

4.8.2 UI Accessibility Guidelines

- The touchscreen interface must follow basic accessibility best practices, including readable font sizes, touch-friendly buttons, and high contrast..

4.8.3 Biomedical Signal Processing Conventions

- Signal processing algorithms must adhere to established EMG feature extraction definitions (RMS, ZC, MNF...) consistent with standards in biomedical literature.

4.8.4 Data Format Standards

- Files used for data loading and saving must follow standard formats supported by NumPy library and plain text for compatibility with external tools.